

Technical Architecture Diagrams

Smart Contract Components and Front-End Integration

Document purpose	High-level technical architecture diagrams compiled from the provided on-chain and off-chain design documents.
Scope	Ask Limit Order flows: Create, Update/Cancel,Buy and My Account
Included layers	Smart contract components, transaction flow, client integration, BFF orchestration, and indexer interactions.

Internal review

Reviewer	Status
<i>Nguyen Hai Chau (Product Lead + Solution Architect)</i>	APPROVED
Do Tran Linh (Project manager)	APPROVED
Tran Anh Quan (<i>Smart Contract Developer</i>)	APPROVED
Truong Quang Khang (Full Stack Developer)	APPROVED
Tran Duc Thang (Full Stack Developer)	APPROVED
Nguyen Thi My Dung (<i>Quality control</i>)	APPROVED

1. Diagram availability in the source materials

The provided documents include both on-chain and off-chain diagrams. The on-chain materials describe the high-level state model, design options, and transaction-level flows for Create, Update/Cancel, and Buy. The off-chain material provides sequence diagrams showing how the client, Kotlin layer, BFF, and indexers interact across Main Market, Upsert PT Ask, Buy PT Ask, and My Account.

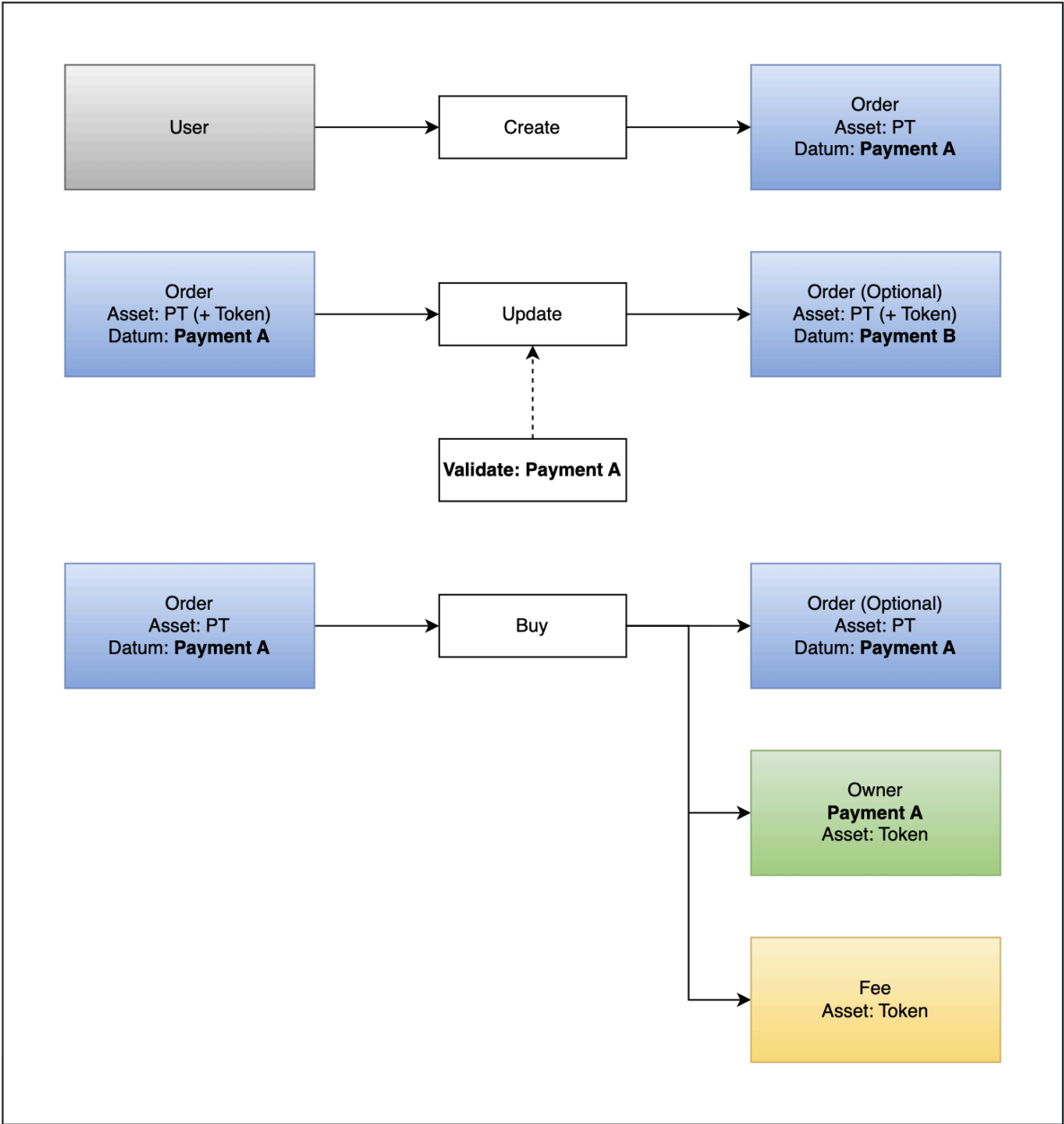
2. Architectural layers covered

The diagrams in this document are organized into two sections. The first section groups smart contract and transaction architecture. The second section groups front-end integration and service orchestration.

3. Smart contract components

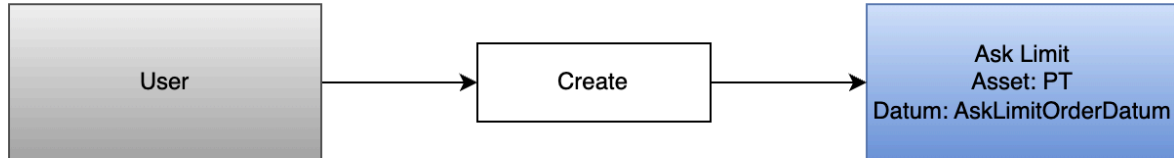
3.1 High-level on-chain state model

The source design describes Ask Limit as the on-chain structure that holds deposited PT and behaves as the seller-side limit order. The lifecycle shown in the source state model moves through create, active, update or redeem, buy, and cancel transitions.



3.2 Transaction flow — Create

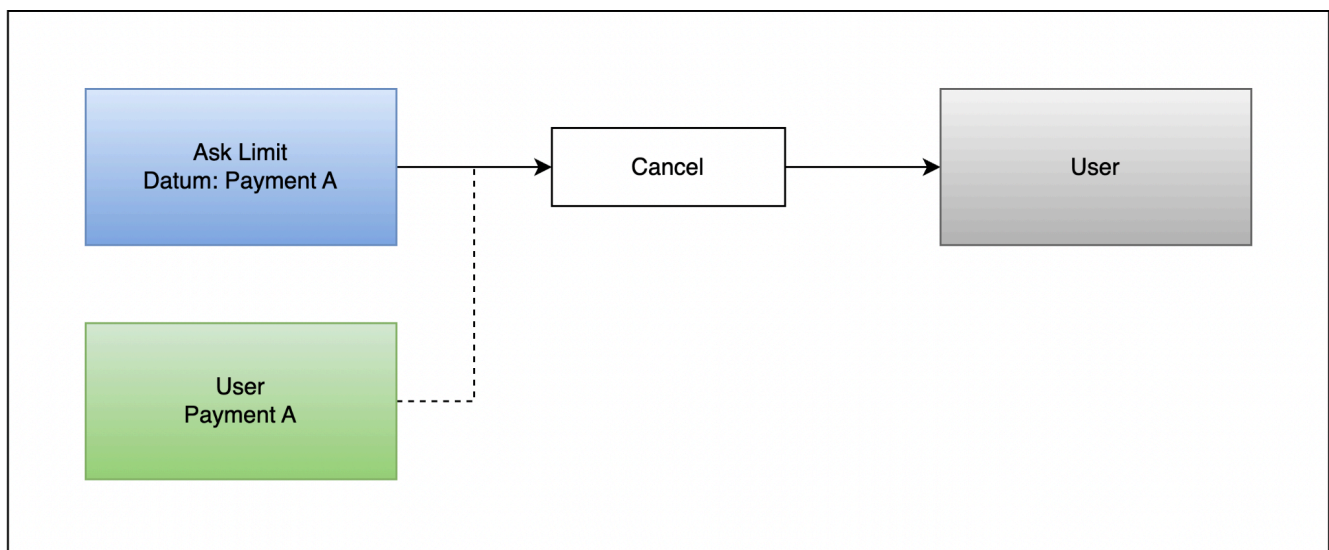
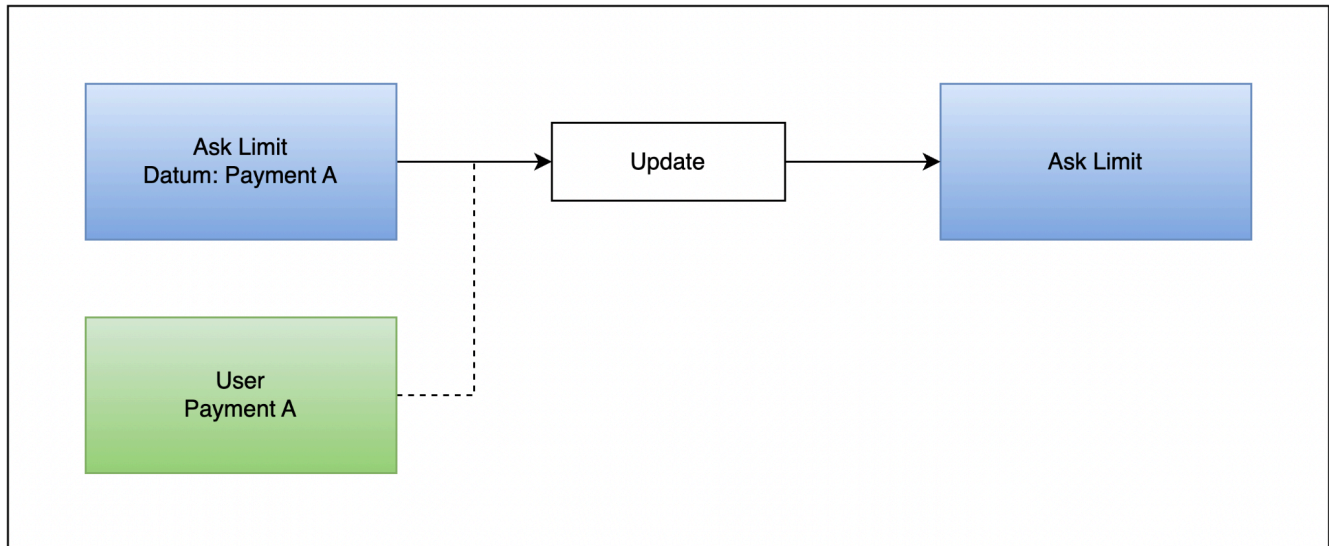
Create is initiated from the user wallet. The source flow shows no script invocation and no reference input. The output is one or more Ask Limit UTxOs holding PT together with the order datum.



1. Signatories:
 - User wallet
2. Script invoked:
 - None
3. Reference Inputs:
 - None
1. Inputs:
 - User UTxO
2. Outputs:
 - **Ask Limit UTxO(s)**

3.3 Transaction flow — Update / Cancel

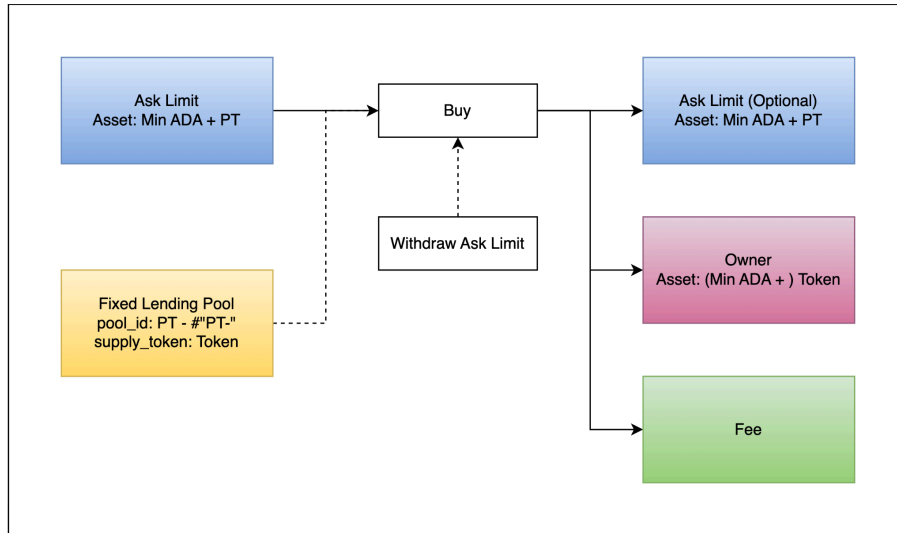
Update and Cancel are controlled through Ask Limit Script Spend. The source design requires the order owner key to be present in the transaction. The output Ask Limit UTxO is optional, depending on whether the order is updated or fully canceled.



1. Signatories:
 - a. User wallet
2. Script invoked:
 - a. **Ask Limit** Script Spend
3. Reference Inputs:
 - a. None
4. Inputs:
 - a. **Ask Limit UTxO**
5. Outputs:
 - a. **Ask Limit UTxO** (optional)
6. Constraint:
 - a. `ask_in_datum.owner_vk`

3.4 Transaction flow — Buy

Buy invokes both Ask Limit Script Spend and Ask Limit Script Withdraw. The source design also uses the Fixed Lending Pool UTxO as a reference input. Depending on fill state, the transaction may produce a continuation Ask Limit UTxO, an owner UTxO, and a fee UTxO.



1. Signatories:
 - a. User wallet
2. Script invoked:
 - a. **Ask Limit Script Spend**
 - b. **Ask Limit Script Withdraw**
3. Reference Inputs:
 - a. **Fixed Lending Pool UTxO**
4. Inputs:
 - a. **Ask Limit UTxO**
5. Outputs:
 - a. **Ask Limit UTxO** (Optional)
 - b. **Owner UTxO**
 - c. **Fee UTxO**
6. Constraint:
 - a. Ask Limit Out
 - i. Address: = Address of **Ask Limit UTxO** input
 - ii. Value:
 1. $\text{ask_out_val.pt} > 0$
 - b. Owner UTxO:
 - i. Address = owner_vk + stake credential of Ask Limit UTxO
 - ii. Value:
 1. origin_token
 - c. Fee UTxO
 - i. Address: = protocol_config.fee_address
 - ii. Value:
 1. origin_token.
 - d. Withdraw:
 - i. Ask Limit

4. Front-end integration

The off-chain source document includes sequence diagrams that connect the user-facing client flow to service orchestration. Across the source flows, the participating components are Bond Dex Client, Bond Dex Client Kotlin, Bond Dex BFF, Fixed Lending Indexer, and Liquid Indexer.

4.1 Main Market

This flow shows how the client loads the main market. The Kotlin layer requests market data from the BFF. When a wallet is connected and the token is ADA, the flow additionally filters bonds in the wallet and requests bond information before mapping the response back to the client.



9:41

Earn 4 Tokens by borrowing every 1,000 ADA per epoch

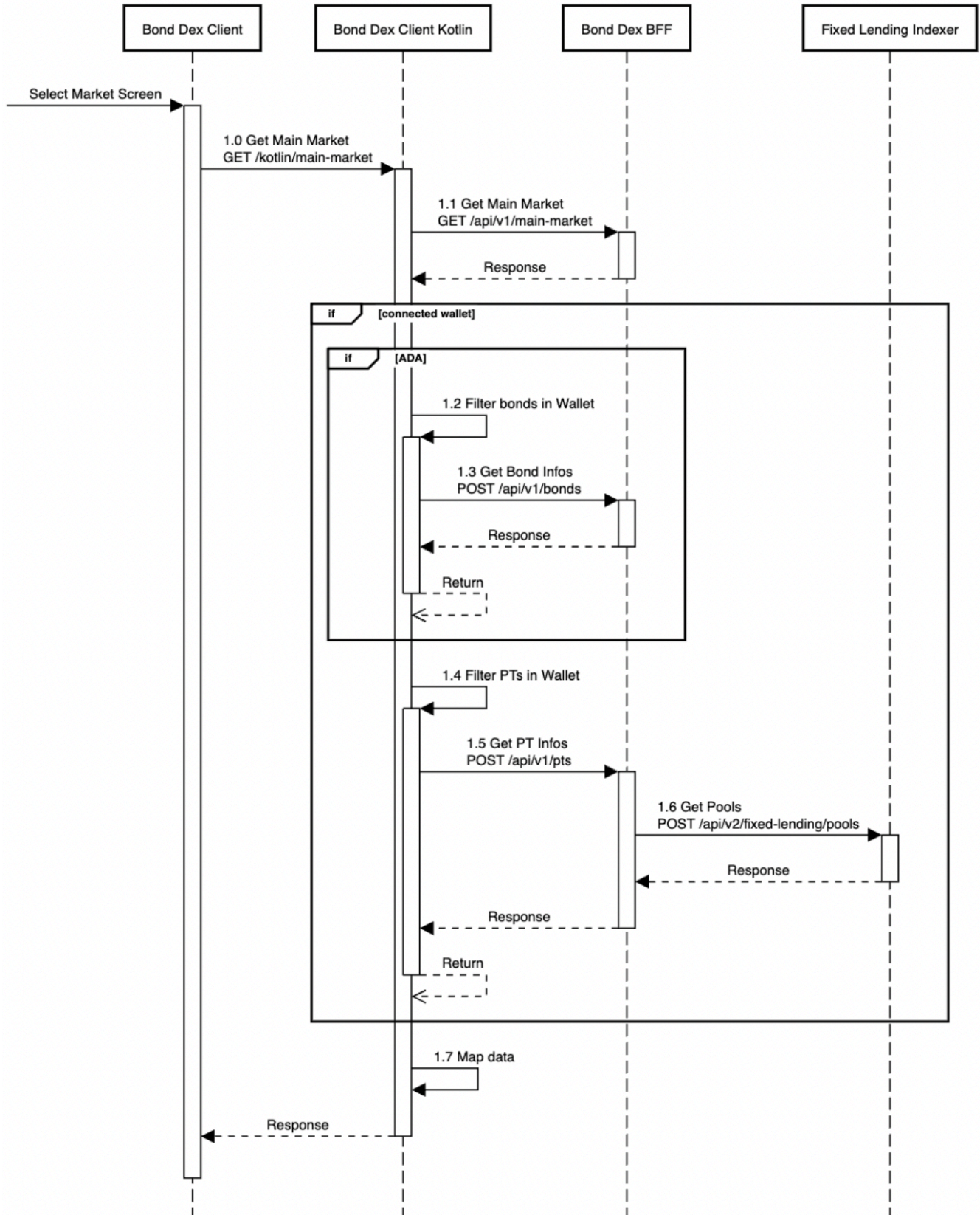
danogo 10,525

Buy PT iUSD Sell PT iUSD

Maturity	Buy Order		Sell Order	
	Volume (iUSD)	Yield	Yield	Volume (PT iUSD)
08-Aug-24	29,970	3.69%	2.09%	500.6M
	748,000	4.06%	2.09%	350.7M
	839,883	4.22%	2.09%	1.345B
28-Aug-24	29,970	3.69%	2.09%	350.5M
02-Sep-24	29,970	3.69%	2.09%	24,000B
07-Sep-24	29,970	3.69%	2.09%	1,098M
12-Sep-24	29,970	3.69%	2.09%	1,000
22-Sep-24	29,970	3.69%	2.09%	10,34K
20-Sep-24	29,970	3.69%	2.09%	34.45

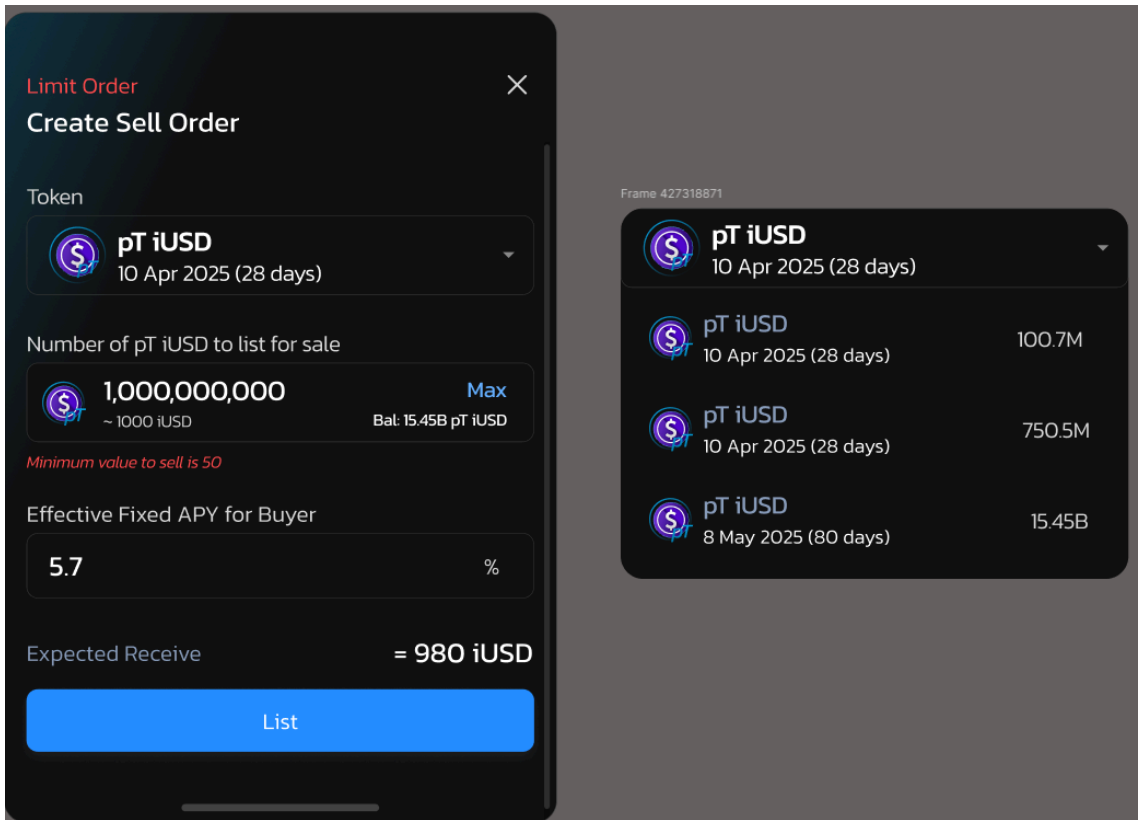
Mainet

Main Screen

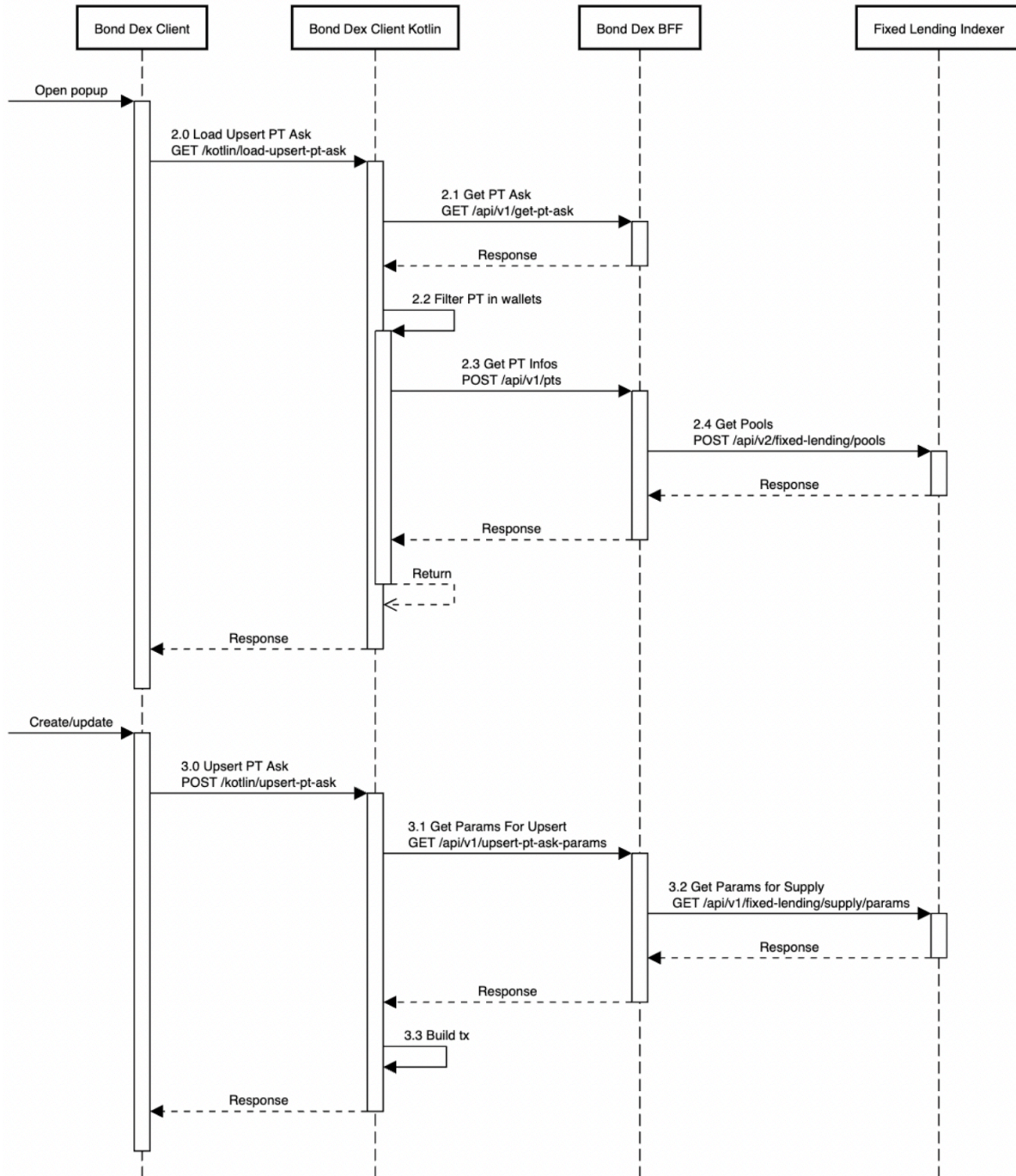


4.2 Upsert PT Ask Create/Update

This flow shows how the create or update popup is loaded and how the transaction-building path is executed. The Kotlin layer loads PT listing data, filters PT in wallets, requests bond information and pool data, then requests upsert transaction parameters before building the transaction.

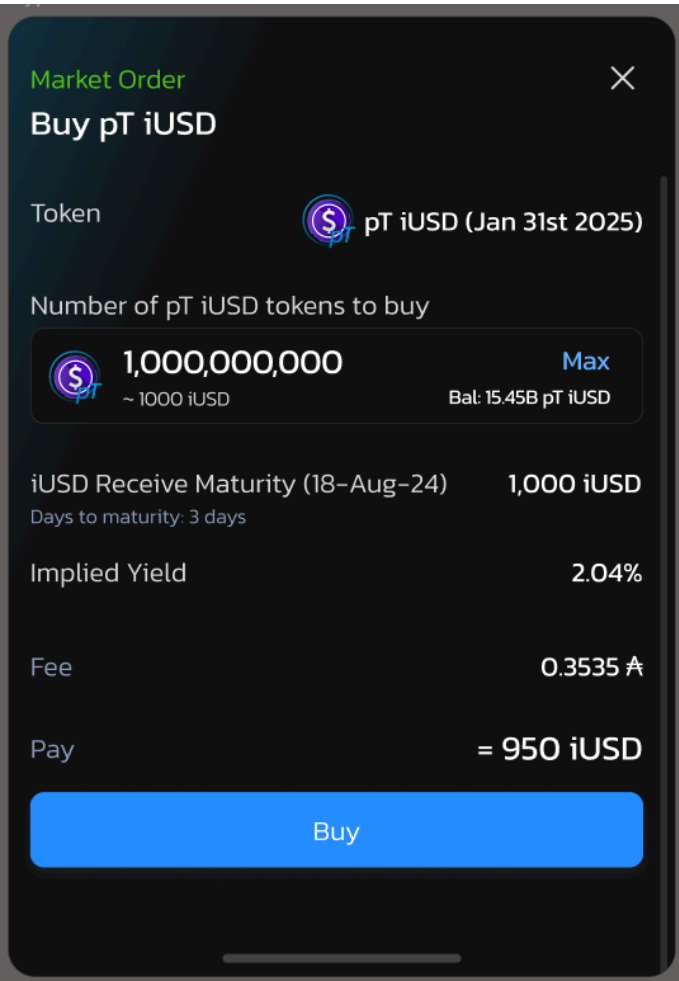


Popup create/update

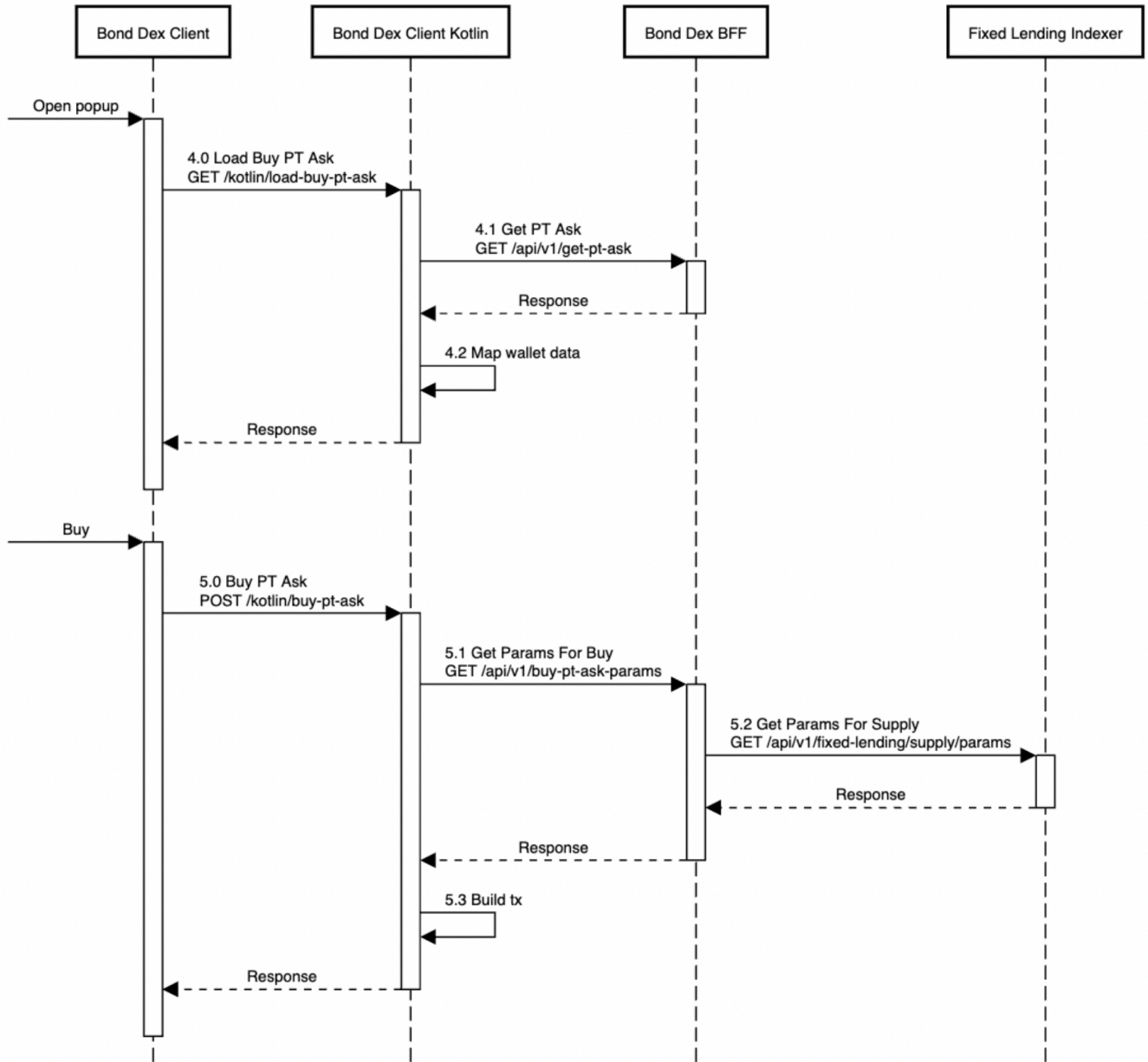


4.3 Buy PT Ask

This flow shows how the buy popup is loaded and how the buy transaction is assembled. The Kotlin layer loads PT ask data, maps wallet data, requests buy parameters, and builds the buy transaction from the returned values.

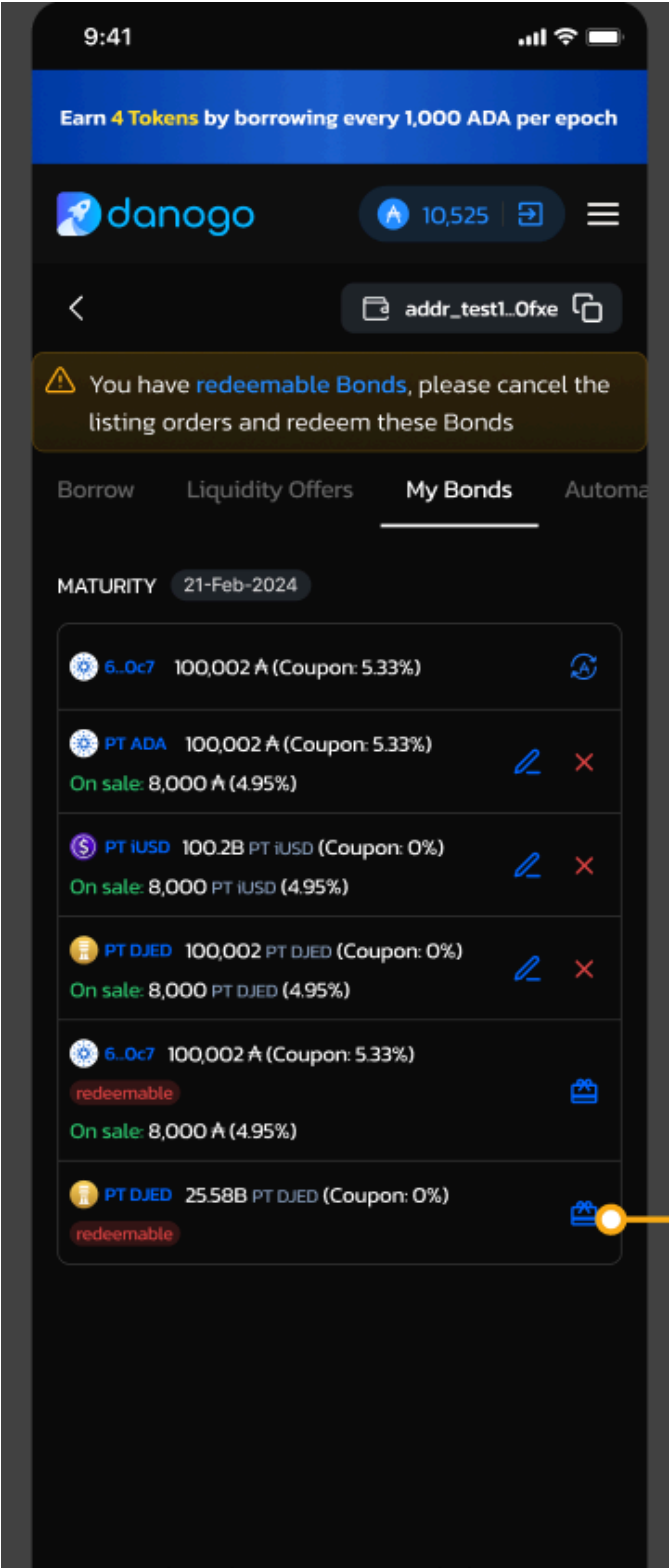


Popup Buy



4.4 My Account

This flow shows account-level retrieval and follow-up actions. The Kotlin layer requests bond market data by address, filters bonds and PTs in the wallet, requests bond information and pools, and supports both cancel and redeem actions through the parameter-loading and transaction-building sequence.



My Account

